

NOTE: This document serves the sole purpose of illustrating my results and documented observations, regarding the VAT Identifier Discovery Task for the Data Assets Intern role at Veridion. I reckon, I used LLMs to optimize and find solutions, but every aspect written below has been personally checked multiple times.

Part 1: Research & Discovery Trails

1. *Domain Discovery: The Missing Link*

The Companies House bulk dataset provides reliable baseline identifiers (Company Name, Number, Status) but lacks a crucial data point for open-web extraction: the company's official website. Without knowing the domain, crawling for VAT numbers is impossible.

Initial Approach: I built a programmatic search module using the duckduckgo-search Python library to query "[Company Name] UK official website". DuckDuckGo was chosen for the Proof of Concept because it does not require paid API keys, allowing for rapid prototyping.

Challenges & Refinements:

2. *SERP Pollution:* Search engine results are heavily polluted by data aggregators and corporate registries (e.g., Companies House, Endole, CompanyCheck, LinkedIn). If the crawler visits these, it will either get blocked or scrape the aggregator's data instead of the target company.
3. *Solution:* I implemented a strict domain-exclusion filter (IGNORE_DOMAINS) to bypass these aggregators and extract the first organic URL.
4. *Rate Limiting:* Querying search engines directly via scripts quickly triggers IP bans (HTTP 429 Too Many Requests).
5. *Mitigation:* For the PoC, I introduced artificial delays (time.sleep) between requests and tested the pipeline on a micro-sample of 20 companies to validate the logic before attempting to scale.

2. DuckDuckGo Search Results & Limitations

- Execution: I ran the domain discovery script on a micro-sample of 20 active companies.
- Result: The script successfully surfaced non-aggregator websites for exactly 5 out of the 20 companies (25% yield).
- Analysis of the Drop-off:

Many UK businesses registered as "Active" simply do not maintain dedicated corporate websites (e.g., local sole traders, shell companies, or small LLCs). They often rely entirely on social media storefronts, which I intentionally excluded to avoid unstructured and non-authoritative scraping.

Search engines often struggle to surface official domains for generic company names without additional geographic context, prioritizing SEO-heavy directory sites instead.

Conclusion on Discovery: Open-web search via free search APIs is a high-latency, low-yield approach. It works for established businesses but leaves a massive gap in coverage. To scale this in production, one would need to cross-reference the Companies House dataset with top-level domain registries (e.g., matching Nominet .co.uk registrations to company names directly) or utilize commercial firmographic datasets.

3. Targeted Scraping & Regex Extraction

- Approach: For the subset of companies with discovered domains, I built a lightweight crawler using requests and BeautifulSoup. To avoid crawling entire domain trees (which is resource-heavy and slow), the PoC evaluates only the homepage index.
- Extraction Logic: UK VAT numbers follow a strict 9-digit structural constraint. I implemented a Regular Expression `((?i)(?:VAT|GB)[^0-9]{0,15}?([0-9]{3}[\s\~]?[0-9]{4}[\s\~]?[0-9]{2}))` designed to capture 9-digit sequences in proximity to keywords like "VAT" or "GB", accounting for arbitrary whitespace or hyphenation often found in human-readable footers.
- Challenges Discovered:
 1. Timeouts & Dead Links: Domains discovered via search engines are not always actively hosted, requiring strict HTTP request timeouts to prevent the crawler from hanging.
 2. Shallow Crawling limits: Many companies hide their VAT number on dedicated "Terms and Conditions" or "Impressum" subpages rather than the homepage, meaning a production-ready crawler must be multi-threaded and capable of traversing internal links, not just the index page.

4. The SEO Pollution Dead End (Search Engine Limitations)

The Expectation: Using a search engine API (DuckDuckGo) with strict query parameters ("{Company Name}" UK official website) and aggregator blacklists would yield genuine corporate homepages for active companies.

The Reality: The crawler failed to extract any VAT numbers because the discovered domains were fundamentally incorrect.

The Evidence: When scanning a subset of 5 allegedly "successful" domain discoveries, the crawler targeted microsoft.com, merriam-webster.com, zhihu.com, and amazon.de.

Why it failed: This exposes a critical flaw in open-web domain discovery for SMEs. When a small UK company (e.g., "GUIDE AND MENTOR LIMITED") lacks a dedicated domain, search algorithms aggressively fall back to keyword matching against ultra-high-authority domains (e.g., matching the word "guide" to a Merriam Webster dictionary page).

Conclusion: Relying purely on NLP/search APIs for domain discovery without a deterministic firmographic database (or cross-referencing domain registry data) introduces catastrophic false-positive domain mappings. This renders downstream VAT scraping useless, as the crawler is scanning irrelevant global domains.

5. Methodology Refinement: Known-Good Data Injection

The Problem: The initial programmatic domain discovery yielded 0% functional, non-aggregator websites for the sample size. Continuing the PoC with this sample would result in a failure to move to the verification phase (HMRC), proving nothing about the technical viability of the scraping and verification modules.

The Decision (Transparency): To validate the complete logic trail (Regex extraction -> HMRC API verification), I made a transparent, methodological decision to inject two known-good entries into the sample_with_domains.csv file. This is common practice in rapid prototyping to test end-to-end integration when up-stream data sources (the domains) fail.

Target Selection Criteria: I chose established UK entities known for high transparency to simulate an ideal scenario for the extractor.

BRITISH COUNCIL (Charity/Global Entity): To test extraction from non-standard (global, complex domain structure) UK entities.

WATERSTONES BOOKSELLERS LIMITED (Retail): To test the ideal "homepage footer" extraction from a large UK retailer.

VAT Identifier Discovery

Anticipated Analysis of Scraper Output:

- **WATERSTONES:** The crawler should easily find the VAT number. UK retail websites are legally required to display it, and Waterstones places it transparently on its homepage, which the PoC extractor targets.
- **BRITISH COUNCIL:** This may fail. As a global charity portal, their homepage is less likely to contain a purely UK identifier. This is a deliberate "known-hard" test case; it will likely require traversing internal links (e.g., to a "Terms" or "Legal" page), proving the limitations of a homepage-only crawler (Part 1 research finding).

Data Integrity & Schema Note (Injected Sample):

- **Observation:** To satisfy the strict schema requirements of the Pandas CSV parser during this test without breaking the pipeline, dummy 8-digit placeholder strings were injected into the CompanyNumber column for both manually introduced entities (Waterstones and British Council).
- **Edge-Case Highlight:** While Waterstones is a standard Private Limited Company and normally possesses a standard 8-digit Companies House number, the British Council operates under a Royal Charter and is registered as a charity. It fundamentally lacks a standard corporate registration number.
- **Production Implication:** This structural anomaly highlights a vital edge-case for the production architecture: a rigid dependency on the Companies House dataset as the sole foundational truth will fail to capture major UK economic actors (charities, royal charters, foreign branches) that do not map to the standard schema, despite them possessing valid VAT registrations.

6. New discoveries. Anti-Bot & Architecture Limits

The Execution: I tested the Regex-based Python crawler against two known-good, highly transparent entities (Waterstones and British Council) to extract their VAT numbers from their respective homepages.

The Result: 0% extraction success.

Why it failed:

1. *Architecture Limits*: Global entities (British Council) do not place local UK tax identifiers on their international landing pages. The data is buried deep within legal sub-directories. A "shallow" crawler (index-page only) is fundamentally insufficient.
2. *Anti-Scraping Defenses*: Large retailers (Waterstones) actively block basic HTTP requests without browser fingerprinting, rendering standard Python scrapers useless against modern WAFs (Web Application Firewalls) like Cloudflare.

Conclusion for Production: A production-ready crawler cannot rely on simple HTTP GET requests to the index page. It requires headless browsers (e.g., Playwright/Puppeteer) to bypass anti-bot challenges and multi-threaded traversal logic to scan specific legal/T&C subpages where VAT numbers legally reside.

7. *Environment Architecture & Dependency Management*

- **Approach**: To ensure the PoC pipeline is fully reproducible and OS-agnostic, I containerized the execution environment using Docker. This prevents host-machine pollution and guarantees consistent execution for the reviewer.
- **Dependency Strategy**: External dependencies required for data manipulation, HTTP requests, and HTML parsing (pandas, requests, BeautifulSoup4) were explicitly managed via a requirements.txt file injected into the container during the build phase.
- **Standard Library Optimization**: To keep the container lightweight and minimize unnecessary third-party bloat, I relied strictly on Python's Standard Library for core operations. For instance, the HMRC validator uses the built-in json module to parse API responses, and the scraper uses the built-in re module for pattern matching, avoiding heavier external frameworks where standard tools suffice.

8. *Data Validation Strategy (API vs. Web UI Scraping)*

- **Approach**: For the final verification module, I intentionally avoided building a scraper over the public "Check a UK VAT number" web interface. Scraping government UI elements is slow, fragile, and highly vulnerable to DOM changes.
- **Implementation**: Instead, the pipeline connects directly to the official HMRC Developer API (api.service.hmrc.gov.uk).
- **Production Value**: This architectural choice guarantees a fast, structured JSON response. By eliminating the need to parse HTML for validation results, this module is deterministic, robust, and immediately ready to scale in a production environment.

Part 3: Scaling with Real Resources

1. Brute-forcing the HMRC Checker via Checksum While generating all mathematically valid 9-digit UK VAT numbers using the modulo-97 checksum algorithm is computationally trivial, pointing this at the HMRC API is a terrible idea for a production system.

Why it fails: It transforms a data engineering pipeline into a Distributed Denial of Service (DDoS) attack. HMRC employs strict rate limiting and monitoring. Blasting their endpoints with millions of generated permutations will result in immediate permanent IP bans and potential legal action. Furthermore, a mathematically valid number is not necessarily an *active* or *assigned* number, making the validation process highly inefficient.

2. Keeping the Dataset Current Companies are dynamic entities. To maintain data freshness without processing the entire bulk dataset daily:

Event-Driven Updates: I would integrate with the Companies House Streaming API, which pushes real-time JSON events when a company registers, dissolves, or updates its details.

TTL (Time-To-Live) on VAT Validation: A successful HMRC validation should have a TTL of 30-90 days. A background cron job would constantly requeue existing valid VAT numbers near their expiration date to verify they haven't been deregistered.

3. Unusable Sources for a Commercial Product I would absolutely not be comfortable selling data sourced from:

Search Engine Scrapes (Google/DuckDuckGo): Beyond the high hallucination rate demonstrated in the PoC, parsing search engine result pages (SERPs) at a commercial scale blatantly violates their Terms of Service, introducing massive legal risk to the core product.